Servomotori Serie M17 - SCHEDA TECNICA

Controllo del movimento tutto-in-uno, semplice ed economico

Dall'istruzione all'innovazione



Introduzione

I servomotori della Serie M17 sono soluzioni di controllo del movimento tutto-in-uno che integrano motore, driver del motore, controllore di movimento ed encoder in un unico contenitore compatto. Questi servomotori dispongono di un'interfaccia di comunicazione RS-485 che consente di collegare più unità in cascata (daisy-chain) e di controllarle da un unico punto di connessione. Disponibile in quattro modelli - M17-60, M17-48, M17-40 e M17-34 - la serie offre opzioni flessibili per soddisfare requisiti specifici di coppia e dimensioni, mantenendo caratteristiche di controllo uniformi. Ogni servomotore della Serie M17 è dotato di funzioni di controllo sofisticate, tra cui molteplici modalità operative, capacità di autocalibrazione e monitoraggio dello stato integrato tramite indicatori LED. I motori possono essere controllati facilmente da qualsiasi piattaforma, inclusi Mac, PC, Raspberry Pi o Arduino (è sufficiente un adattatore RS485 economico), risultando ideali sia per applicazioni didattiche sia industriali. La serie supporta il controllo preciso della posizione con profili di movimento trapezoidali, la modalità di controllo ad anello chiuso e una gestione completa degli errori. Grazie alle dimensioni di montaggio standardizzate NEMA 17, all'ampio intervallo di tensione (12-24V) e a un protocollo di comunicazione robusto, i servomotori della Serie M17 offrono una soluzione affidabile e flessibile per le applicazioni che richiedono un controllo preciso del movimento: dalla robotica alle macchine CNC, dalle apparecchiature di collaudo automatizzate agli strumenti scientifici.

Inviateci il vostro feedback

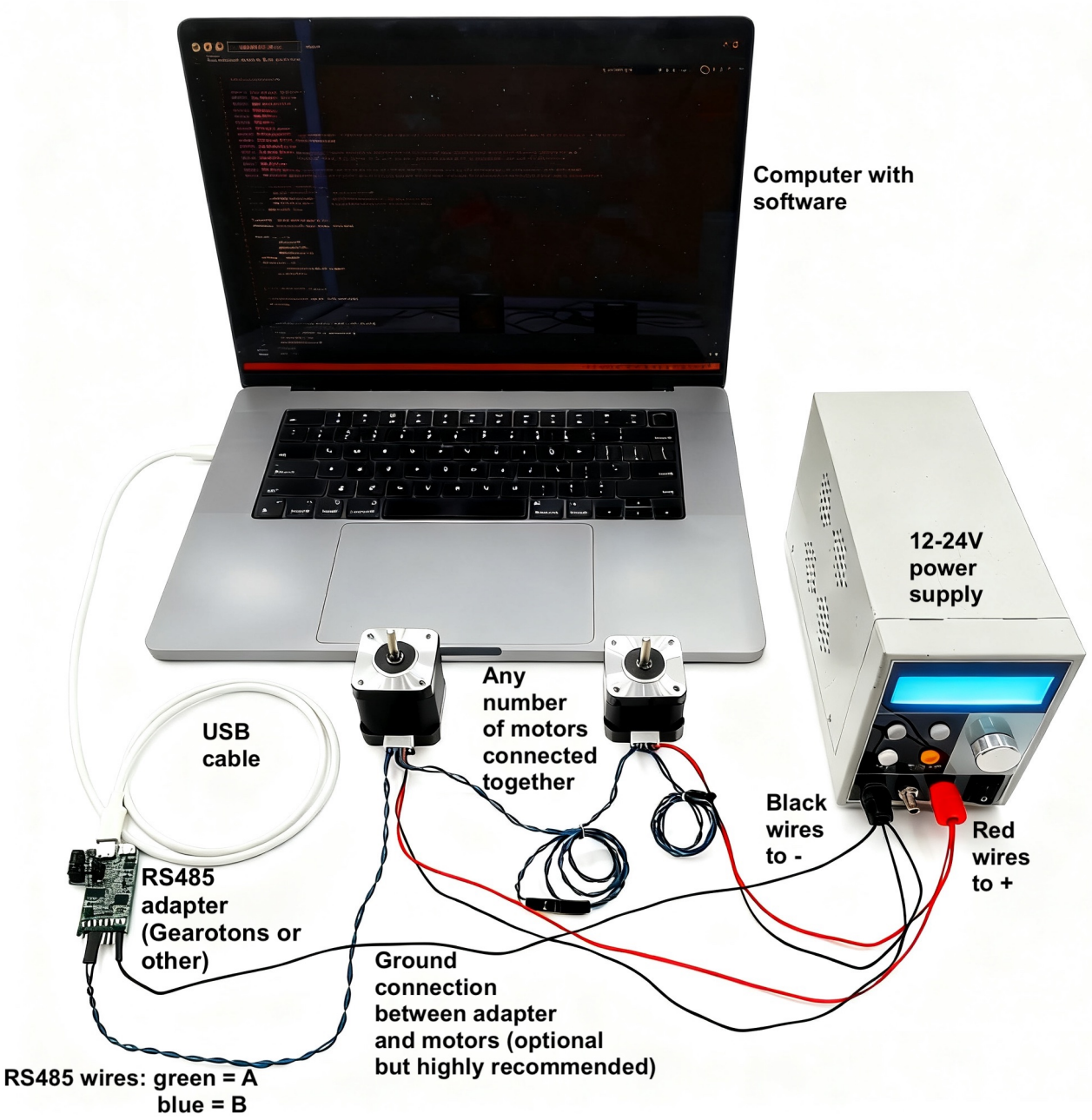
Se riscontrate errori in questo documento o avete domande, segnalatecelo all'indirizzo: tutorial.gearotons.com/feedback. Apprezzeremmo molto i vostri suggerimenti su come migliorare il prodotto o la documentazione. Grazie mille!

Visitate la nostra pagina di feedback

Caratteristiche principali

- Elevato livello di integrazione: motore, driver del motore, sistema di controllo del movimento ed encoder in un unico dispositivo
- Controllo di un numero qualsiasi di motori da un unico semplice controllore tramite interfaccia di comunicazione RS-485
- Controllo tramite comandi di alto livello come "Enable mosfets" (abilita i MOSFET) e "Trapezoid move" (movimento trapezoidale), evitando il controllo DIR/STEP con temporizzazioni critiche
- Formato compatto, di dimensioni quasi identiche a quelle di un motore passo-passo NEMA 17 con le stesse specifiche (senza sporgenze)
- Dimensioni di montaggio standardizzate NEMA 17
- Ampio intervallo di tensione (12-24V) per opzioni di alimentazione flessibili
- Controllo ad anello chiuso ad alta precisione con encoder integrato e anello di controllo PID eseguito a 32 kHz
- Efficienza energetica nettamente superiore nel controllo ad anello chiuso rispetto a un motore passo-passo convenzionale
- Protezione integrata da sovracorrente, sovratensione e sovratemperatura
- Velocità massima fino a 560 RPM
- Rapporto coppia/peso identico a quello di un motore passo-passo equivalente
- Compatibile con un'ampia varietà di interfacce e hardware, come Raspberry Pi, Arduino, ESP32, Mac e PC
- Ci impegniamo a fornire documentazione e tutorial eccellenti per rendervi operativi rapidamente
- Forniamo documentazione adatta all'IA, nel caso vogliate lasciare che la vostra IA preferita faccia tutto il lavoro al posto vostro
- Adatto a robotica, CNC, automazione, strumenti scientifici, attrezzature di collaudo, stampanti 3D e molto altro
- Disponibile in diverse taglie, per trovare la coppia e il prezzo giusti per la vostra applicazione

Schema di collegamento



Sistema di unità di misura

I servomotori della Serie M17 utilizzano determinate unità interne per eseguire in modo efficiente i calcoli associati al movimento (mediante aritmetica intera). Spetta al software di controllo supportare più unità di misura per le varie grandezze. Le nostre librerie Python e Arduino gestiscono automaticamente le conversioni di unità, permettendovi di lavorare con le unità che preferite. Di seguito sono riportate le unità supportate per ciascuna grandezza:

Grandezza	Unità disponibili
Tempo	seconds, milliseconds, minutes, microseconds, timesteps
Posizione	shaft rotations, degrees, radians, encoder counts
Velocità	rotations per second, rpm, degrees per second, radians per second, counts per second, counts per timestep
Accelerazione	rotations per second squared, rpm per second, degrees per second squared, radians per second squared, counts per second squared, counts per timestep squared
Corrente	internal current units, milliamps, amps
Tensione	volts, millivolts
Temperatura	celsius, fahrenheit, kelvin

Guida introduttiva

Per aiutarvi a iniziare con il vostro servomotore della Serie M17, mettiamo a disposizione una guida online completa che copre ogni aspetto, dalla configurazione iniziale alle implementazioni avanzate del protocollo. Questa guida include:

- Istruzioni di configurazione passo-passo
- Documentazione dettagliata del protocollo di comunicazione
- Esempi di programmazione e frammenti di codice
- Descrizione dei codici di errore
- Consigli per la risoluzione dei problemi e buone pratiche



Fate clic qui per la guida introduttiva: tutorial.gearotons.com

LED indicatori e pulsanti

Il servomotore dispone di due LED di stato (verde e rosso). Il LED verde lampeggia lentamente per indicare il normale funzionamento (heartbeat), lampeggia rapidamente per indicare che e' in esecuzione il bootloader anziche' l'applicazione e si accende brevemente durante la ricezione di un pacchetto, tremolando quindi in presenza di comunicazione sul bus. Il LED rosso indica i codici di errore fatale lampeggiando un determinato numero di volte.

Il servomotore dispone di due pulsanti denominati "Reset" e "Test". Il pulsante Reset reimposta il microcontrollore interno e riporta tutto lo stato ai valori predefiniti. Il pulsante Test fa ruotare il motore. Premere brevemente per farlo ruotare in un senso; premere per più di 0,3 secondi e rilasciare per farlo ruotare nel senso opposto. Tenere premuto per almeno 2 secondi e rilasciare per portare il motore in modalità ad anello chiuso. Tenere premuto per più di 15 secondi e rilasciare per far eseguire al motore un'autocalibrazione. Si noti che durante la calibrazione il motore ruota e deve poter ruotare liberamente affinché la calibrazione riesca: rimuovere quindi eventuali carichi prima di eseguire questa operazione.

Panoramica del protocollo di comunicazione

I servomotori della Serie M17 utilizzano RS-485 per la comunicazione, consentendo di collegare più motori in cascata (daisy-chain). Ogni motore ha un indirizzo univoco a 64 bit assegnato in fabbrica, che ne permette l'indirizzamento individuale su un bus RS-485. A ogni motore può inoltre essere assegnato un alias univoco di 1 byte (0-251), per risparmiare byte nel pacchetto di comunicazione mantenendo un controllo individuale analogo. È disponibile anche un alias di broadcast, il 255. Il protocollo supporta un insieme completo di comandi per il controllo del movimento, la configurazione e il monitoraggio dello stato. È supportato l'aggiornamento del firmware tramite l'interfaccia RS-485. La velocità di comunicazione è fissa a 230400 baud.

Riepilogo di riferimento dei comandi

Per la fonte aggiornata e autorevole di tutti i comandi disponibili, consultare questo documento.

https://github.com/tomrodinger/servomotor/blob/main/python_programs/servomotor/motor_commands.json

È inoltre possibile eseguire questo comando:

```
pip3 install servomotor   # run this just once to install the library and programs
servomotor_command -c
```

Questo comando stampa le informazioni contenute nel file motor_commands.json in un formato più leggibile e fornisce alcune indicazioni d'uso per inviare comandi al motore dalla riga di comando.

Controllo di base

Comando	Descrizione
Disable MOSFETs	Disabilita immediatamente le uscite del driver del motore (i MOSFET); il comando viene eseguito subito (non e' accodato) ed e' idempotente.
Enable MOSFETs	Abilita le uscite del driver del motore (i MOSFET).
Reset time	Azzerare l'orologio assoluto in microsecondi del dispositivo; come importante effetto collaterale, l'intera coda dei movimenti viene svuotata e il motore si arresta immediatamente.
Emergency stop	Arrestare immediatamente il motore: disabilitare i MOSFET, svuotare la coda dei movimenti, azzerare la velocità corrente e impostare il riferimento interno di posizione sulla posizione corrente.
Zero position	Imposta la posizione corrente come posizione zero (l'origine).
System reset	Reimposta il dispositivo.

Controllo del movimento

Comando	Descrizione
Trapezoid move	Percorre lo spostamento con segno indicato esattamente nella durata specificata, relativamente alla posizione al termine dei movimenti precedentemente accodati (per un target assoluto utilizzare 'Go to position').
Go to position	Accoda un movimento assoluto fluido.
Homing	Esegue l'azzeramento (homing) del motore muovendosi fino a incontrare un arresto fisico.
Go to closed loop	Entra nella modalita' di controllo di posizione ad anello chiuso.
Move with acceleration	Accoda un segmento ad accelerazione costante: l'accelerazione viene applicata una volta per passo temporale per il numero di passi temporali indicato.
Move with velocity	Fa ruotare il motore a velocita' costante per una durata prefissata.
Multimove	Accoda fino a 32 movimenti in un unico comando; ogni movimento e' un segmento di accelerazione o una variazione istantanea di velocita' (selezionata per ciascun movimento dalla bitmask moveTypes) eseguito per il numero di passi temporali indicato.

Configurazione

Comando	Descrizione
Set maximum velocity	Imposta la velocita' massima consentita.
Set maximum acceleration	Imposta l'accelerazione massima usata per pianificare tutti i movimenti trapezoidali accodati successivamente ('Go to position', 'Trapezoid move', movimenti di calibrazione); gli elementi gia' presenti in coda non ne sono influenzati.
Start calibration	Esegue una calibrazione completa del motore.
Set maximum motor current	Imposta la corrente massima del motore e la corrente massima di rigenerazione.
Set safety limits	Imposta i limiti di sicurezza di posizione.
Test mode	Imposta o attiva una modalita' di test.
Set PID constants	Imposta le costanti P, I e D del controllore ad anello chiuso che cerca di mantenere la traiettoria di movimento.
Set max allowable position deviation	Impostare lo scostamento massimo consentito tra la posizione reale del motore (misurata dai sensori Hall) e quella comandata; il superamento genera l'errore fatale 45 (ERROR_POSITION_DEVIATION_TOO_LARGE).
CRC32 control	Abilita o disabilita il livello CRC32 del protocollo per questo dispositivo, in entrambe le direzioni.
Get PID constants	Leggere un'istantanea coerente delle costanti P, I e D grezze configurate.

Stato e monitoraggio

Comando	Descrizione
Get current time	Restituire il tempo assoluto del dispositivo come numero senza segno a 64 bit di microsecondi reali trascorsi dall'accensione o dall'ultimo 'Reset time'; non è l'ora civile.
Get n queued items	Restituire il numero di elementi nella coda dei movimenti come singolo byte tra 0 e 32 (la coda contiene al massimo 32 elementi).
Get hall sensor position	Restituisce la posizione misurata dai sensori Hall, ovvero la posizione effettiva dell'albero.
Get status	Restituisce i flag di stato del motore e il codice di errore fatale.
Control hall sensor statistics	Attiva o disattiva la raccolta delle statistiche dei sensori Hall.
Get hall sensor statistics	Legge le statistiche raccolte dai tre sensori Hall analogici.
Get position	Restituisce la posizione comandata (desiderata) corrente, ovvero la posizione target interna del profilo di movimento che avanza a ogni tick di controllo.
Get comprehensive position	Restituisce in un'unica operazione la posizione comandata, la posizione (misurata) dei sensori Hall e il conteggio dell'encoder esterno.
Get supply voltage	Restituisce la tensione misurata dell'alimentazione.
Get max PID error	Restituisce l'errore di posizione minimo e massimo osservato dal controllore PID ad anello chiuso dall'ultima lettura, quindi azzera gli accumulatori.
Get temperature	Restituisce la temperatura misurata da un sensore analogico dedicato sul PCB del driver del motore (non il die del microcontrollore e non gli avvolgimenti del motore).
Get debug values	Leggere un'istantanea di numerosi valori diagnostici interni.
Get communication statistics	Restituisce sei contatori di errori di comunicazione e, facoltativamente, li azzera.

Gestione del dispositivo

Comando	Descrizione
Time sync	Inviare la lettura dell'orologio del master al motore affinché possa regolare la propria frequenza di clock.
Get product specs	Restituisce due costanti di prodotto necessarie per i calcoli di movimento.
Detect devices	Rileva tutti i dispositivi collegati all'interfaccia RS485.
Set device alias	Imposta l'alias di un byte del dispositivo, lo salva nella memoria flash non volatile e quindi reimposta automaticamente il dispositivo.
Get product info	Restituisce le informazioni sul prodotto: codice prodotto (numero di modello), codice di compatibilita' del firmware, versione hardware, numero di serie e ID univoco, restituiti come payload di risposta di 28 byte.
Firmware upgrade	Scrive nella flash una pagina di 2048 byte del firmware applicativo.
Get product description	Restituisce una breve stringa di descrizione del prodotto.
Get firmware version	Restituisce la versione del firmware oppure la versione del bootloader se il dispositivo sta attualmente eseguendo il bootloader.
Ping	Invia un payload di 10 byte e il dispositivo risponde immediatamente con gli stessi 10 byte (viene eseguito subito, non accodato).
Vibrate	Far vibrare il motore (implementato solo su M1; senza effetto su M2, M17 e M23) alternando rapidamente la tensione PWM ad anello aperto tra +50 e -50 (ampiezza fissa codificata), oppure arrestare la vibrazione.
Identify	Identifica un motore facendo lampeggiare rapidamente il suo LED verde: 30 lampeggi di circa 90 ms ciascuno, per un totale di circa 2,7 secondi, dopodiche' riprende il normale lampeggio heartbeat di 1 secondo.

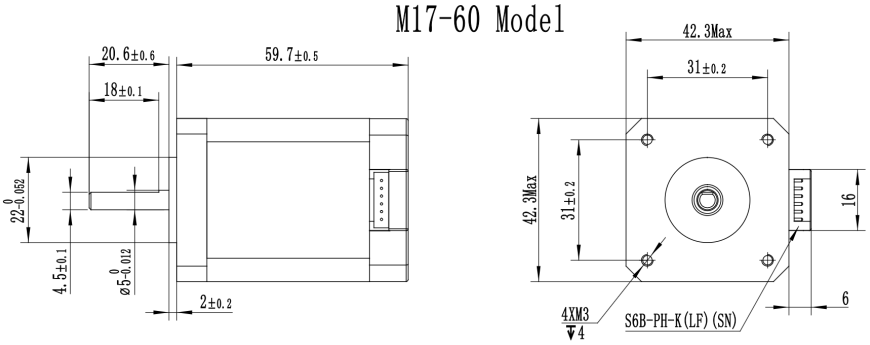
Altro

Comando	Descrizione
Capture hall sensor data	Acquisisce i dati dei sensori Hall e li ritrasmette in streaming.
Read multipurpose buffer	Legge e cancella il buffer dati multiuso, che contiene al massimo un set di dati alla volta.

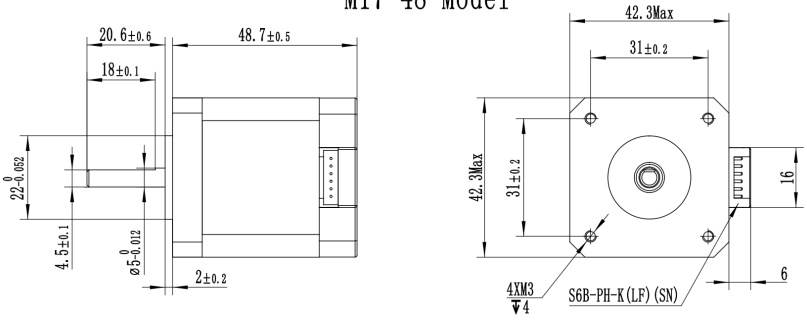
Specifiche meccaniche

Parametro	M17-60	M17-48	M17-40	M17-34
Dimensioni (LxP)	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm
Altezza	59.7 mm	48.7 mm	40.1 mm	33.5 mm
Lunghezza albero	20.6 mm	20.6 mm	20.6 mm	20.6 mm
Peso	470g	360g	285g	210g
Grado di protezione	IP20	IP20	IP20	IP20

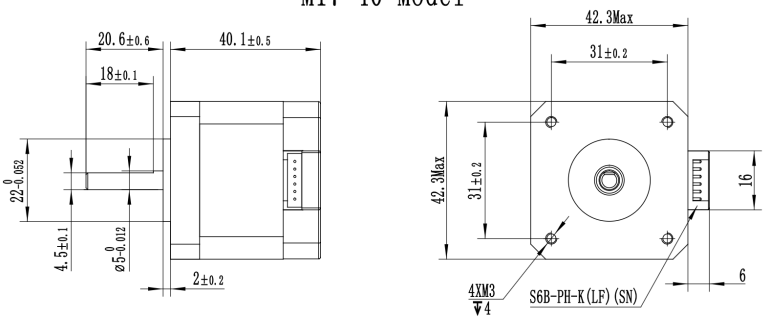
M17-60 Model



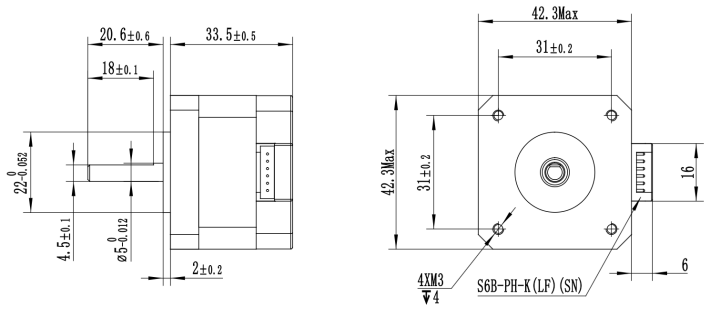
M17-48 Model



M17-40 Model



M17-34 Model



Specifiche tecniche

Parametro	M17-60	M17-48	M17-40	M17-34
Tensione di esercizio	12-24V	12-24V	12-24V	12-24V
Coppia nominale	0.65 N.m	0.55 N.m	0.42 N.m	0.28 N.m
Velocità massima	560 RPM	560 RPM	560 RPM	560 RPM
Corrente massima	1.1A	1.1A	1.1A	1.0A
Potenza nominale	26.4W	26.4W	26.4W	24.0W

Condizioni di esercizio

Parametro	Specifica
Temperatura di esercizio	da 0C a +80C
Temperatura di immagazzinamento	da -20C a +60C
Intervallo di umidità	dal 20% all'80% RH (senza condensa)
Ambiente di installazione	Solo per uso interno

Libreria Python

La nostra libreria Python fornisce un'interfaccia completa per il controllo dei servomotori della Serie M17. La libreria gestisce tutti i protocolli di comunicazione di basso livello e le conversioni di unità, semplificando l'integrazione del controllo del servomotore nei vostri progetti Python.

Installazione

Installare la libreria servomotor utilizzando pip:

```
pip3 install servomotor
```

Esempio: controllo di base del motore

L'esempio seguente mostra come connettersi a un servomotore, abilitare i MOSFET ed eseguire un movimento trapezoidale:

```
#!/usr/bin/env python3
"""
Minimal trapezoid move: rotate 1 turn in 1 second.
Edit ALIAS below if needed. Uses rotations and seconds.
"""
import time, servomotor
from servomotor import communication

# Hard-coded settings for a minimal demo
ALIAS = 'X'                               # Device alias, change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Serial device path; change if needed (e.g., "COM3" on
                                          #  Windows)
DISPLACEMENT_ROTATIONS = 1.0              # 1 rotation
DURATION_SECONDS = 1.0                    # 1 second
DELAY_MARGIN = 0.10                       # +10% wait margin because the motor's clock is not
                                          #  perfectly accurate

communication.serial_port = SERIAL_PORT # if you comment this out then the program
                                          #  should prompt you for the serial port or it will use
                                          #  the last used port from a file
servomotor.open_serial_port()

m = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations", verbose=0)
m.enable_mosfets()
m.trapezoid_move(DISPLACEMENT_ROTATIONS, DURATION_SECONDS)
time.sleep(DURATION_SECONDS * (1.0 + DELAY_MARGIN))
m.disable_mosfets()

servomotor.close_serial_port()
```

Caratteristiche principali

- Supporto di tutti i comandi del motore
- Conversione automatica delle unità per tempo, posizione, velocità, accelerazione, corrente, tensione e temperatura
- Gestione degli errori e monitoraggio dello stato
- Supporto di più motori sullo stesso bus
- Diventate operativi con poche righe di codice

Documentazione

Per la documentazione completa delle API e ulteriori esempi, visitare:
https://github.com/tomrodinger/servomotor/tree/main/python_programs

Libreria Arduino

La libreria Arduino per i servomotori della Serie M17 fornisce un'interfaccia di facile utilizzo per il controllo dei motori dalle schede Arduino. La libreria supporta la comunicazione seriale hardware e include tutte le funzioni essenziali di controllo del motore.

Installazione

Installare la libreria tramite il Gestore librerie di Arduino:

- 1. Aprire l'IDE di Arduino
- 2. Andare su Sketch → Includi libreria → Gestisci librerie
- 3. Cercare "Servomotor"
- 4. Individuare la libreria denominata "Servomotor by Gearotons" e fare clic su Installa



Installazione manuale

In alternativa, se siete esperti, potete scaricare il codice della libreria Arduino qui: https://github.com/tomrodinger/Servomotor_Arduino_Library

Esempio: controllo di base del motore

Questo esempio mostra come inizializzare un motore, abilitare i MOSFET ed eseguire un movimento trapezoidale:

```
// Minimal Arduino example: Trapezoid move using built-in unit conversions
// Goal: spin the motor exactly 1 rotation in 1 second, then stop.
// Sequence:
//  system reset -> enable MOSFETs -> trapezoidMove(1.0 rotations, 1.0 seconds)
//  -> wait 1.1s -> disable MOSFETs.
//
// Notes:
// - This uses the library's unit conversion (no raw counts/timesteps).
// - Configure Serial1 pins for your board (ESP32 example pins below).
// - Motor is created AFTER Serial1.begin(...) so hardware UART pins are set first.
// - Every command is checked with getError(). No command method returns a success
//   flag, and a motor that has latched a fatal error silently ignores everything
//   afterwards, so an unchecked sketch just stops moving with no symptom at all.
//   See "Checking for errors" in the API documentation.

#include <Servomotor.h>

#define ALIAS 'X'                   // Device alias
#define BAUD 230400                 // RS485 UART baud rate
#define DISPLACEMENT_ROTATIONS 1.0f // 1 rotation
#define DURATION_SECONDS 1.0f       // 1 second
#define TOLERANCE_PERCENT 10        // +10% wait margin because the motor's clock is not
                                    //  perfectly accurate
#define WAIT_MS ((unsigned long)(DURATION_SECONDS * 1000.0f * (100 + TOLERANCE_PERCENT) / 100))
#define POST_RESET_WAIT_MS 1500     // Keep the bus SILENT this long after a reset: the motor
                                    //  boots through a bootloader window, and any packet that
                                    //  arrives during it pins the motor in the bootloader.

// Example RS485 pins for ESP32 DevKit (change as needed for your board)
#if defined(ESP32)
#define RS485_TXD 4             // TX pin to RS485 transceiver
#define RS485_RXD 5             // RX pin from RS485 transceiver
#endif

// Returns true (and explains itself) if the previous command did not succeed.
// getError() is 0 on success, positive for a fatal error reported by the motor
// (look the number up in the Error Codes section of the documentation), and
// negative for a communication failure such as -1 = no reply within 1 second.
bool failed(Servomotor &motor, const char *what) {
  int e = motor.getError();
  if (e == 0) return false;
  Serial.print("[FAIL] ");
  Serial.print(what);
  Serial.print(" -> getError() = ");
  Serial.print(e);
  Serial.println(e > 0 ? "  (motor fatal error - see the Error Codes section)"
                       : "  (communication failure - check wiring, alias and power)");
  return true;
}

void setup() {
  Serial.begin(115200); // Console serial for debugging
                        // On ESP32-S3, set Tools > USB CDC On Boot > Enabled or this
                        // output never reaches the USB serial monitor.

  // Create the motor; serial port opens on first instantiation.
#if defined(ESP32)
  Servomotor motor(ALIAS, Serial1, RS485_RXD, RS485_TXD);
#else
  Servomotor motor(ALIAS, Serial1);
#endif

  // Use units: rotations for position, seconds for time.
  // These are host-side only - they send nothing to the motor.
  motor.setPositionUnit(PositionUnit::SHAFT_ROTATIONS);
  motor.setTimeUnit(TimeUnit::SECONDS);

  // Start from a known-clean state: this also clears any fatal error left over
  // from a previous run, which is the usual reason a motor "stops working".
  motor.systemReset();
  if (failed(motor, "systemReset")) return;
  delay(POST_RESET_WAIT_MS);

  motor.enableMosfets();
```

```
  if (failed(motor, "enableMosfets")) return;

  motor.trapezoidMove(DISPLACEMENT_ROTATIONS, DURATION_SECONDS);
  if (failed(motor, "trapezoidMove")) return;

  delay(WAIT_MS);

  // A successful move command only means the move was accepted and queued. Faults
  // such as a stall or a position-deviation trip happen later, while it executes,
  // so check the motor's own status once the motion should be finished.
  getStatusResponse status = motor.getStatus();
  if (failed(motor, "getStatus")) return;
  if (status.fatalErrorCode != 0) {
    Serial.print("[FAIL] motor faulted during the move, error code ");
    Serial.println(status.fatalErrorCode);
    return;
  }

  motor.disableMosfets();
  if (failed(motor, "disableMosfets")) return;

  Serial.println("Move completed successfully.");
}

void loop() {
}
```

La nostra missione

In Gearotons crediamo che il futuro appartenga a chi comprende i sistemi di intelligenza artificiale e di automazione ed è in grado di lavorare al loro fianco. La nostra missione è dare forza alla prossima generazione di innovatori, rendendo la tecnologia avanzata di controllo del movimento accessibile, comprensibile ed economica per maker, educatori e ingegneri di tutto il mondo. Ci impegniamo ad abbattere le barriere che hanno tradizionalmente separato studenti e hobbisti dagli strumenti di automazione di livello professionale. Offrendo soluzioni integrate che eliminano la complessità senza sacrificare le funzionalità, rendiamo possibili esperienze di apprendimento pratico che preparano le giovani menti a un mondo in cui la creatività umana e la precisione delle macchine lavorano in armonia. La nostra convinzione fondamentale è che ogni studente debba avere l'opportunità di sperimentare con gli elementi costitutivi dell'automazione moderna: dalla robotica ai sistemi CNC, fino agli strumenti scientifici. Attraverso le nostre collaborazioni con il mondo dell'istruzione e il nostro approccio open source, stiamo formando una generazione che non si limita a usare la tecnologia, ma la comprende e la plasma davvero. Insieme stiamo costruendo le fondamenta per gli innovatori che definiranno uno straordinario futuro.

Open Source

Crediamo nel rendere il mondo migliore attraverso la tecnologia. Il software, il firmware e i file di progetto dei PCB sono disponibili qui:

https://github.com/tomrodinger/servomotor

Versione della scheda tecnica: 1.11 Data di rilascio: Oct. 29, 2026

(c) 2026 Tutte le specifiche sono soggette a modifica senza preavviso.